

A Lean Standards-Aware HTTP/1.1 Server in Haskell: Implementation and Tail-Latency Evaluation under Open-Loop Load

Henry George Grantham-Smith

MSci Computer Science

University of Exeter

Abstract

This dissertation presents the design and evaluation of a lean HTTP/1.1 server implemented directly in Haskell. It asks whether a small, standards-aware server can remain competitive with established implementations when judged by tail latency rather than peak throughput alone. The server supports a scoped but non-trivial subset of HTTP/1.1, including persistent connections, request framing, filesystem-backed resources, conditional requests, and byte-range handling, and is validated using a socket-level end-to-end test suite.

Performance is evaluated using a reproducible benchmark harness built around fixed-rate open-loop load generation and a shared byte-identical payload corpus. The custom server is compared with Warp, Go net/http, nginx, and Flask served through Gunicorn across three workloads: a 1 KiB JSON response, a 50 KiB binary payload, and a 1 MiB binary payload.

For the JSON workload, and for typical 50 KiB runs after accounting for an explicit outlier, the custom server remains in the same broad p99 latency regime as Warp, Go, and nginx while sustaining the intended offered load. For the 1 MiB workload, however, tail latency deteriorates markedly despite still meeting the target rate. A targeted diagnostic pass using cost-centre and heap profiling, with RTS event logs retained as supporting artefacts, strengthens the interpretation that the weakness lies primarily in response emission and response-path allocation rather than repeated disk I/O.

The dissertation argues that this negative result is valuable: a modest Haskell HTTP/1.1 server can be correct and competitive for small and medium responses, but large-payload tail behaviour remains the decisive engineering challenge.

Declaration

I declare that this report is my own work except where explicitly acknowledged through references. Any material derived from external sources has been identified accordingly.



Signature

Contents

1	Introduction	3
2	Background and Related Work	3
3	Aims, Scope, and Success Criteria	4
4	Design	5
4.1	Architectural overview	5
4.2	Connection lifecycle and concurrency model	5
4.3	Request parsing, target validation, and connection semantics	5
4.4	Request-body framing and limits	6
4.5	Resource model and protocol support	6
4.6	Conditional requests and range handling	7
4.7	Why these design choices suit the evaluation	7
5	Implementation and Testing	7
5.1	Code organisation	7
5.2	Benchmark workload implementation	9
5.3	Testing strategy	9
5.4	What the tests do and do not establish	10
6	Experimental Methodology	11
6.1	Benchmark harness and baseline set	11
6.2	Load points and run configuration	12
6.3	Open-loop load generation	13
6.4	Repetition, order rotation, and aggregation	13
6.5	Reproducibility measures	14
6.6	Targeted diagnostic pass	14
6.7	Threats to validity	15
7	Results	15
7.1	Overall picture	17
7.2	Interpreting the Flask baseline	18
7.3	JSON workload	18
7.4	50 KiB workload	18
7.5	1 MiB workload	19
7.6	Targeted diagnostic pass	20
8	Discussion and Critical Reflection	21
8.1	What the successful cases really show	21
8.2	Interpreting the 1 MiB weakness from the implementation	22

8.3	Why the 50 KiB outlier matters methodologically	22
8.4	Suitability of Haskell for this task	22
8.5	Limitations of the present design	23
8.6	Future work	23
9	Conclusion	24
A	Raw benchmark tables	26
B	Diagnostic evidence	32

1 Introduction

HTTP servers sit at an awkward intersection of protocol law and systems performance. On one side, RFC 9110 and RFC 9112 impose concrete obligations on request parsing, message framing, connection reuse, and error handling; on the other, real deployments are judged as much by latency under load as by formal correctness [2, 3]. The project reported here grew out of that tension. It began with a simple question: if the abstractions of a mature framework are stripped away and the protocol is implemented directly over sockets in Haskell, how much performance can be recovered without retreating to a benchmark-only toy?

That question is worth asking precisely because Haskell web servers are no longer a novelty. Marlow’s early account of a concurrent Haskell web server already showed that a conforming and performant design was possible, and later systems such as Warp established Haskell as a viable platform for serious web infrastructure [5, 9]. What remains less well explored is the narrower issue of tail behaviour under disciplined measurement. Much community benchmarking still privileges throughput or median latency, even though modern systems work has repeatedly shown that a small fraction of slow responses can dominate user experience and system capacity [1, 7].

In this dissertation, tail latency means latency at high percentiles of the response-time distribution, especially p95, p99, and p99.9. A p99 latency of 40 ms means that 99% of observed requests completed within 40 ms, while the slowest 1% took longer. This matters because end users and downstream services often experience systems through their slowest interactions rather than through their average ones: a server with excellent median latency can still feel unreliable if a small but persistent fraction of requests stalls under load.

This dissertation therefore does not attempt to prove that Haskell can host web servers in general; that point has long been settled. Its aim is more specific. A lean HTTP/1.1 server is built as an experimental instrument, then compared with Warp and a small set of non-Haskell baselines under fixed-rate open-loop load. The goal is to determine where a compact design remains competitive, where it does not, and what that says about the practical limits of this implementation strategy.

The central research questions are as follows.

- **RQ1.** Can a lean HTTP/1.1 server implemented directly over sockets in Haskell provide correct behaviour for a non-trivial subset of protocol semantics?
- **RQ2.** Under fixed-rate open-loop load, how does its tail latency compare with Warp, Go `net/http`, nginx, and Flask+Gunicorn across small and large response workloads?
- **RQ3.** What do the observed results imply about the practicality and limits of this design approach?

These questions are answered using two forms of evidence: a socket-level test suite for protocol behaviour, and a controlled benchmarking harness designed to expose tail effects rather than suppress them.

2 Background and Related Work

The project sits within three strands of prior work.

First, HTTP/1.1 semantics matter because they determine what it means for a server to be “real” rather than synthetic. Persistent connections, the `Host` header requirement, message delimitation through `Content-Length` or chunked transfer encoding, conditional requests, and range responses all affect server state and resource use in ways that can influence latency as well as correctness [2, 3]. Restricting the feature set is reasonable in a dissertation project, but only if the retained subset still

captures the protocol decisions that make request handling non-trivial.

Second, there is a substantial Haskell lineage behind this work. Marlow’s *Developing a High-Performance Web Server in Concurrent Haskell* is historically important because it demonstrated that lightweight concurrency and careful runtime support made network servers a serious Haskell systems problem rather than an academic curiosity [5]. Warp later pushed that point much further, combining GHC lightweight threads, `ByteString`-centred data paths, and streaming response construction into the dominant WAI backend for Haskell web applications [9, 11, 12]. Work on GHC’s scalable I/O manager and later runtime experiments such as Mio also matters here, since a Haskell server’s latency profile is inseparable from scheduler behaviour, garbage collection, and the efficiency of the runtime’s evented I/O path [6, 8].

Third, the methodology literature changes how such a system should be judged. Dean and Barroso’s account of “the tail at scale” explains why high-percentile latency matters even when averages look acceptable [1]. Tene’s critique of coordinated omission makes the practical benchmarking point sharper: closed-loop load generators can systematically under-report bad latency by allowing the system under test to throttle demand implicitly [7, 13]. For the present project, this means that a benchmark reporting only requests per second, or even median latency, would answer the wrong question.

These strands suggest a clear gap. There is ample evidence that Haskell can support performant servers, and ample advice on how latency should be measured, but relatively little work that combines a standards-aware HTTP/1.1 implementation with a modern, tail-focused comparative study. That gap is the space this dissertation addresses.

The contribution of this dissertation is therefore not to show that Haskell can host web servers in general; that has already been established by systems such as Warp. Instead, it asks a narrower and more practically useful question: where does a deliberately lean, standards-aware implementation remain competitive when evaluated using tail-latency-first methodology rather than throughput alone? By combining a direct socket-level implementation, open-loop benchmarking designed to avoid coordinated omission, and targeted diagnostic profiling of large-response weakness, the project shifts the focus from feasibility to boundary conditions: not whether Haskell can support web servers, but where this implementation strategy stops being competitive and why.

3 Aims, Scope, and Success Criteria

The server developed in this project is intentionally narrow in scope. It supports HTTP/1.1 over plain TCP, persistent connections, a selected set of request methods, request-body framing through `Content-Length` and `Transfer-Encoding: chunked`, and filesystem-backed routes with conditional and range semantics. TLS termination, HTTP/2, HTTP/3, middleware ecosystems, and general application-framework concerns are out of scope.

This scoping reflects a methodological choice rather than a convenience. The aim is to preserve the parts of HTTP/1.1 that make server behaviour meaningful, while keeping the implementation small enough that design decisions remain explicit and discussable. Success in that context is not “beating every baseline”. A more defensible set of criteria is:

1. the custom server should pass a broad socket-level test suite covering the scoped protocol features;
2. the benchmarking process should be reproducible and fair, using shared payload bytes, fixed offered load, repeated runs, and recorded environment metadata;
3. the server should remain close to the strongest comparable baselines on at least some workloads, especially the smaller responses;

4. where it is not competitive, the report should explain that failure plainly rather than writing around it.

In this dissertation, “competitive” is used in a bounded comparative sense: remaining in the same broad tail-latency regime as the strongest comparable baselines at the selected offered loads, rather than matching or beating them universally.

The final criterion prevents the evaluation from becoming a success-only narrative. A negative result is often more informative than a flattering but weakly supported claim.

4 Design

4.1 Architectural overview

The custom server is implemented directly over sockets in Haskell. A listening socket is opened with `SO_REUSEADDR`, each accepted connection is handed to a new lightweight thread via `forkFinally`, and the handler then loops over that connection until the negotiated preference switches to close. This arrangement follows the style that Marlow and later Warp work make plausible in Haskell: the code reads like a thread-per-connection server, while the GHC runtime is responsible for multiplexing those lightweight threads efficiently [5, 6, 9].

The design is intentionally explicit at the level this dissertation is concerned with. Request-head parsing, target validation, body framing, body consumption, response construction, benchmark payload management, and filesystem semantics are split into separate modules rather than hidden behind a framework abstraction. That separation is not only a software-engineering preference. It makes the system easier to reason about: when a design trade-off is discussed, the corresponding code path is visible and small enough to inspect directly. This does not mean that every scheduling or I/O multiplexing decision is controlled by hand. The deliberate boundary is that protocol behaviour remains explicit in the code, while low-level scheduling and readiness management are delegated to the GHC runtime.

4.2 Connection lifecycle and concurrency model

Connection handling begins in `acceptLoop`, which accepts a socket and immediately spawns `handleConn`. Within a connection, the server retains any unread bytes left after request-head parsing and feeds them into the next request cycle. This matters for both persistent connections and pipelined requests. The implementation is therefore not limited to “one request per connection” benchmarking; it supports keep-alive and is able to consume a second request already present in the residual buffer.

Given that boundary, the concurrency model is deliberately uncomplicated. There is no application-level worker pool, custom accept queue, or hand-written event loop. Instead, the implementation exposes a simple thread-per-connection structure and relies on GHC’s lightweight-thread scheduler and I/O manager for multiplexing. The main virtue of this choice is interpretability. A more elaborate design might have improved some operating points, but it would also have made it harder to distinguish protocol costs from scheduling policy. For the current project, delegating this part of the design to the runtime is an acceptable trade rather than an accidental omission.

4.3 Request parsing, target validation, and connection semantics

Request heads are parsed incrementally using `attoparsec` [10]. The parser recognises the request line, HTTP/1.0 and HTTP/1.1 versions, and up to 100 headers. Header names are restricted to

`tchar`-style field names, and the outer read loop imposes a total request-head budget of 8192 bytes with a five-second receive timeout. These numbers are implementation policy choices rather than constants prescribed by HTTP/1.1. Their role is to place finite bounds on input that would otherwise be unbounded: the protocol permits recipients to impose practical limits on what they are willing to parse, provided over-limit or incomplete messages are handled as HTTP request failures rather than by blocking indefinitely [2, 3].

Target handling is wider than the benchmark endpoints alone require. The implementation distinguishes origin-form, absolute-form, authority-form, and the `*` form used by `OPTIONS`. Origin-form and absolute-form requests are normalised to a path; authority-form is only accepted for `CONNECT`, where it currently returns `501 Not Implemented` rather than pretending to tunnel; invalid or misplaced target forms are rejected with `400 Bad Request`. Host validation is likewise stricter than many toy servers: duplicate `Host` headers are rejected, bracketed IPv6 literals and explicit ports are accepted, and HTTP/1.1 requests without a valid host are refused.

Connection preference is derived from protocol version plus the `Connection` header. HTTP/1.1 defaults to keep-alive unless `close` is present; HTTP/1.0 defaults to close unless `keep-alive` is requested. This logic is conventional, but important, because it shapes how the server behaves under repeated requests on the same socket and therefore affects the realism of the evaluation.

4.4 Request-body framing and limits

Body framing is handled separately from request-line parsing. If `Transfer-Encoding` and `Content-Length` are both present, the request is rejected. That strictness is not just defensive programming: HTTP/1.1 treats ambiguous framing metadata as a security-sensitive case because disagreement about message length is a classic route to request smuggling [3]. Within the supported implementation subset, chunked request bodies are decoded, while unsupported or ambiguous transfer-coding cases are rejected rather than partially interpreted. If only `Content-Length` is present, duplicated values must agree exactly, otherwise the request is rejected. This is a sensible subset of RFC behaviour for a dissertation system: it covers the important ambiguity cases without claiming full support for every possible transfer-coding combination.

Body consumption also has explicit limits. The body reader caps bodies at 16 MiB, enforces a five-second receive timeout, and constrains a chunk-size line to 8192 bytes. Again, the values are not HTTP-standard values. They are operational limits chosen for this implementation. The relevant standards relationship is categorical rather than numeric: a server may decide that a request body is larger than it is willing to process, corresponding to the condition represented by `413 Content Too Large`, and a server that has not received a complete request within the time it is prepared to wait is in the territory represented by `408 Request Timeout` [2]. Fixed-length bodies are read exactly; chunked bodies are decoded strictly, with trailers drained before the connection buffer is returned to the main loop. The server therefore avoids shortcutting all non-trivial bodies into a benchmark-only path.

The implementation also supports `Expect: 100-continue`, but only where it is meaningful. For HTTP/1.1 requests whose target is known to consume a body, the server emits the interim `100 Continue` response before reading the payload. For HTTP/1.0, the same header is ignored.

4.5 Resource model and protocol support

The project contains two distinct resource families. The benchmark endpoints `/json`, `/file50k`, and `/file1m` return deterministic preloaded payloads held in memory. They exist to isolate transport and response costs from application logic. Alongside them, a filesystem-backed route family under `/fs/` exposes a broader slice of HTTP semantics: `GET`, `HEAD`, `PUT`, `POST`, and `DELETE`; conditional

requests through entity tags and modification dates; byte-range responses; and create-or-replace versus create-only semantics.

This wider protocol surface is important. A server that only returns a fixed payload tells us little about the cost of being standards-aware. Here, benchmark serving and semantic coverage are separated cleanly: the benchmark endpoints keep the measurements controlled, while the filesystem routes demonstrate that the implementation does more than generate synthetic responses.

Path handling for filesystem routes is conservative. Requests are accepted only beneath `/fs/`, path segments are split and validated manually, and empty segments, `.`, `..`, and NUL bytes are rejected. That keeps traversal behaviour explicit and testable.

4.6 Conditional requests and range handling

Precondition handling is implemented directly in the server rather than delegated to middleware. `If-Match` uses strong comparison; `If-None-Match` uses weak comparison; `If-Modified-Since` and `If-Unmodified-Since` compare normalised HTTP dates; and the outcome is reduced to one of three cases: pass, not modified, or failed precondition. This design keeps the decision logic separate from the file-operation code that follows it.

Entity tags are generated from the normalised modification time and file size, and range handling currently supports a single `bytes=` range rather than arbitrary multi-range responses. That is a reasonable limit. Single-range support is sufficient to exercise the key semantics without forcing the server into multipart boundary generation, which would add complexity without helping the benchmark study.

4.7 Why these design choices suit the evaluation

Taken together, these choices produce a system that is lean but not empty. The server is small enough that design decisions remain inspectable, yet wide enough in protocol surface that the benchmark results are attached to a system with meaningful HTTP behaviour. Using a full framework would have hidden the protocol decisions, while a tiny benchmark responder would not have been a serious HTTP implementation.

5 Implementation and Testing

5.1 Code organisation

Table 1 summarises the main implementation units. It aligns the narrative of the report with the organisation of the codebase, which in turn makes the technical claims easier to verify.

Table 1: Main implementation units and their responsibilities.

Component	Role	Why it matters in the report
<code>Server.hs</code>	Listening socket setup, accept loop, per-connection loop, route dispatch, filesystem semantics, conditional requests, and range handling.	Concentrates the protocol decisions that determine whether the server is merely functional or genuinely HTTP-aware.
<code>Http/Parse.hs</code>	Incremental request-head parsing, target-form interpretation, host validation, and connection preference logic.	Makes parser and target-handling choices explicit rather than hidden inside a framework.
<code>Http/Framing.hs</code>	Request-body framing decisions for <code>Content-Length</code> , <code>Transfer-Encoding</code> , and <code>Expect</code> .	Separates message-body correctness from route handling.
<code>Http/Body.hs</code>	Exact fixed-length reads, strict chunked decoding, trailer draining, timeouts, and body-size limits.	Defines how body framing behaves under malformed or incomplete input.
<code>Http/Response.hs</code>	Status values and raw response construction, including <code>Content-Length</code> logic.	Keeps the wire-level output path compact and inspectable.
<code>Workloads.hs</code>	Deterministic generation and loading of the 1 KiB, 50 KiB, and 1 MiB benchmark payloads.	Ensures the benchmark corpus is reproducible and identical across implementations.
<code>app-warp/Main.hs</code>	Minimal Warp baseline serving the same in-memory payloads.	Provides the closest in-language comparison point.
<code>test/Spec.hs</code>	Socket-level end-to-end test suite.	Grounds claims about correctness in observable protocol behaviour.
<code>scripts/*.sh</code> , <code>collect_diagnostics.sh</code> , and <code>summarize_trimmed.py</code>	Build, benchmark orchestration, metric extraction, diagnostic collection, run ordering, and trimmed aggregation.	Makes both the main benchmark session and the later profiling/eventlog pass reproducible rather than ad hoc.

A later diagnostic extension also added lightweight instrumentation around the benchmark endpoints and a dedicated collection script for event-log and profiling artefacts. Those additions are not used to generate the headline comparison tables, but they matter because they support the explanation of the 1 MiB weakness with profiling evidence rather than purely inferred from code inspection.

For reproducibility, the codebase is organised so that the main executable lives under `app/`, the Warp baseline under `app-warp/`, core server modules under `src/`, and the socket-level tests under `test/`. Benchmark orchestration lives under `scripts/`, generated comparison baselines under `comparison_baselines/`, and archived benchmark outputs under `bench/` and `bench/diagnostics/`. The body figures and tables are derived from the raw and aggregated benchmark data reported in Appendices A and B, so the main evidence used for the dissertation’s claims is contained within the report itself rather than depending on ad hoc terminal output.

5.2 Benchmark workload implementation

The benchmark payloads are not generated on demand. They are loaded once at startup by `Workloads.loadBenchPayloads`, which also rewrites the on-disk files if they differ from the deterministic corpus expected by the project. The JSON payload is padded to exactly 1024 bytes so that the small-response workload has a fixed and auditable size rather than whatever length happens to result from a compact JSON literal. The padding is therefore a benchmarking control, not a claim about typical JSON document structure. It fixes `Content-Length`, makes the payload byte-identical across implementations, and prevents small accidental size differences from contaminating latency comparisons. The binary files are generated from deterministic repeating patterns to exactly 50 KiB and 1 MiB respectively.

The same corpus is mirrored into the comparison baselines. The harness records SHA-256 checksums for the Haskell-side files and the baseline copies, and refuses to proceed if the corpora are out of sync. Without this control, the apparent gap between implementations could partly reflect serving different bytes.

5.3 Testing strategy

Correctness is evaluated through a live socket-level test suite rather than by unit testing parser helpers alone. The test executable launches the server on an ephemeral port, sends requests over real sockets, and checks status lines, headers, bodies, connection closure, and read behaviour. This is materially stronger evidence than a set of narrow parser tests because it exercises the protocol surface the way a client sees it.

The suite contains 53 end-to-end checks. The number is less important than the coverage pattern. Simple success cases are present, but so are the awkward cases that commonly reveal shortcuts: duplicated or invalid `Host` headers, conflicting framing metadata, short bodies, malformed chunked input, absolute-form requests, authority-form misuse, pipelined requests, conditional updates, and range negotiation.

Table 2: Test coverage categories in the socket-level suite.

Category	Representative checks	Purpose
Basic semantics	GET /, HEAD /, /json, /file50k, /file1m, Connection: close, and pipelined requests.	Verifies the ordinary request-response path and connection reuse behaviour used in the benchmark workloads.
Framing and bodies	Fixed-length echo, chunked echo, Expect: 100-continue, invalid or duplicated Content-Length, conflicting Transfer-Encoding plus Content-Length, short bodies, and malformed chunked bodies.	Checks that the server does real message framing rather than treating request bodies as a side concern.
Target and host validation	Missing host, duplicate host, invalid host, host with explicit port, bracketed IPv6 host, absolute-form requests, non-CONNECT authority-form requests, and CONNECT returning 501.	Ensures that request-target interpretation and host handling are consistent with the scoped HTTP behaviour.
Filesystem operations	PUT, POST, DELETE, path traversal rejection, and 204 No Content behaviour.	Demonstrates that the server handles a broader route family than the benchmark endpoints alone.
Conditional and range semantics	Last-Modified, ETag, If-Modified-Since, If-Unmodified-Since, If-Match, If-None-Match, range success and failure, If-Range, and HEAD plus range interaction.	Makes the claim of standards awareness credible by testing the non-trivial parts of resource semantics.

5.4 What the tests do and do not establish

The test suite gives good evidence for the scoped semantics, but it does not prove full HTTP conformance. It is targeted rather than exhaustive, and it is written by the same author as the server. The point of the suite is not to certify the server against every corner of RFC 9110 and RFC 9112; it is to show that the project’s claims about the supported subset are backed by live behavioural evidence.

The phrase “standards-aware” is therefore used deliberately rather than “standards-compliant”. The implementation follows selected HTTP/1.1 requirements where they are within scope, especially around request targets, message framing, connection reuse, conditional requests, and byte ranges, but it has not been validated against an independent HTTP conformance suite and should not be read as a complete implementation of RFC 9110 or RFC 9112.

That still matters a great deal for the benchmark interpretation. It is much easier to trust a latency comparison when the server under test has already been forced through malformed input, connection reuse, and conditional-update cases that would quickly expose a benchmark-only stub.

6 Experimental Methodology

6.1 Benchmark harness and baseline set

The benchmarking process is driven by shell scripts rather than by manual command lines. The main session runner builds the custom executable and Warp baseline, provisions the comparison baselines, verifies corpus parity, rotates benchmark order across repetitions, and records session logs. This infrastructure is mundane, but it is exactly the sort of infrastructure that makes a performance report credible.

The baseline set mixes like-for-like and illustrative comparisons. Warp is the closest peer, since it shares the Haskell runtime and serves the same in-memory payloads through WAI. Go `net/http` provides a strong non-Haskell application-server reference. nginx is valuable as a static-serving reference, but should not be over-read as a direct application-server peer. Flask behind Gunicorn is retained as a deliberately simple Python/WSGI application-stack baseline so that the harness includes an obviously weaker regime and not only a cluster of strong implementations. It is therefore useful as an interpretive contrast, but not as a tuned representation of all Python web deployment.

Table 3 makes the benchmark environment explicit, while Tables 4 and 5 summarise how each baseline was launched and what was or was not tuned. This does not remove every reproducibility limit, but it closes a gap in the earlier draft by making the machine, topology, toolchain, and comparison setup directly auditable.

Table 3: Experimental setup used for the final benchmark session.

Host machine	Single local Linux Mint 22 Cinnamon system used for both load generation and serving.
CPU	AMD A10-7800 Radeon R7 APU; 4 CPU cores were used in the recorded benchmark session.
RAM	Approximately 3.3 GiB visible to the OS during the recorded session.
Storage	1001.7 GB local storage; storage medium type not separately recorded.
OS / desktop / kernel	Linux Mint 22 Cinnamon; Cinnamon 6.2.9; Linux kernel 6.8.0-106-generic.
Client/server topology	<code>wrk2</code> and all compared servers ran on the same host over loopback addresses <code>127.0.0.1:8080-8084</code> .
Network path	Loopback only; no external NIC path or real network traversal was included in the reported measurements.
Haskell toolchain	Stack 3.7.1 and GHC 9.10.3.
RTS configuration	Main Haskell benchmark executables were built with <code>-threaded -rtsopts -with-rtsopts=-N</code> . No additional RTS flags were passed during the main comparative benchmark session.
Diagnostic RTS configuration	The eventlog diagnostic reruns used the normal executable with event logging enabled: <code>+RTS -N -l-au -RTS</code> . The profiled diagnostic reruns used a cost-centre-profiled executable with heap profiling enabled: <code>+RTS -N -p -hc -i0.01 -RTS</code> . Eventlog runs used the same 60 s benchmark duration as the main session; profiled runs used 30 s to limit profiling overhead.
Load generator	<code>wrk</code> 4.0.0 [epoll] with 2 client threads in the final harness.
Recorded comparison tool versions	nginx 1.24.0, Go 1.22.2, Python 3.12.3, Flask 3.1.3, and Gunicorn 25.1.0.

Table 4: Baseline launch configuration used by the benchmark harness.

Implementation	Version	Launch command	Process / thread configuration
Lean server	custom build	<code>stack exec - webserver-exe</code>	1 process; GHC lightweight threads; RTS default <code>-N</code> .
Warp	same toolchain as lean build	<code>stack exec - warp-baseline</code>	1 process; GHC lightweight threads; RTS default <code>-N</code> .
Go <code>net/http</code>	Go 1.22.2	<code>go run .</code>	1 process; Go runtime goroutines.
nginx	1.24.0	<code>nginx -p \$PWD -c nginx.rendered.conf -g 'daemon off;'</code>	1 worker process.
Flask + Gunicorn	Flask 3.1.3; Gunicorn 25.1.0	<code>./run_gunicorn.sh</code>	1 worker; <code>sync</code> ; 1 thread; <code>--preload</code> .

Table 5: Baseline keep-alive, logging, and payload configuration.

Implementation	Keep-alive	Logging	Payload source / notes
Lean server	supported; not separately overridden in the harness	stdout/stderr redirected to session logs; no separate access log configured	Deterministic payloads preloaded from <code>bench_files/</code> ; benchmark endpoints served from memory.
Warp	default HTTP/1.1 behaviour; not separately tuned	stdout/stderr redirected to session logs; no separate access log configured	Uses the same <code>Workloads.loadBenchPayloads</code> path and the same in-memory benchmark corpus.
Go <code>net/http</code>	default library behaviour; not separately tuned	stdout/stderr redirected to session logs; no per-request access log configured	Loads the copied benchmark corpus from <code>comparison_baselines/bench_files/</code> at startup and serves those bytes directly.
nginx	<code>keepalive_ timeout65;</code>	<code>access_logoff;</code>	Serves the copied benchmark corpus as static files; <code>sendfileon</code> ; in the generated configuration.
Flask + Gunicorn	not separately tuned in the launcher	access log enabled via <code>--access-logfile-</code>	Flask app reads the copied benchmark corpus at startup and serves those preloaded bytes.

The client/server topology was intentionally local and controlled. The `wrk2` client and all compared servers ran on the same host and communicated over loopback rather than across a physical network. This was chosen to keep the comparison tightly controlled and to minimise environmental variance between implementations, but it also means that the reported measurements include same-host contention and do not directly characterise real network traversal.

6.2 Load points and run configuration

The final benchmark session uses fixed operating points rather than a full saturation sweep. That is a limitation, though not fatal, as the dissertation’s main question concerns specifically tail latency under selected offered load rather than maximum throughput.

The workload configuration is shown in Table 6. Each case is run for 60 seconds with two `wrk2` threads. The connection counts differ slightly by workload, with 100 connections for the JSON and

50 KiB cases and 50 connections for the 1 MiB case.

Table 6: Benchmark operating points used in the final session. The selected points are intentionally fixed operating regimes rather than a full load sweep.

Workload	Endpoint	Offered load	Connections	Duration	Reason for inclusion
1 KiB JSON	<code>/json</code>	2000 requests/s	100	60 s	Small in-memory API-style benchmark response with minimal payload movement.
50 KiB binary	<code>/file50k</code>	300 requests/s	100	60 s	Medium response that remains memory-resident but moves enough data to reveal buffering effects.
1 MiB binary	<code>/file1m</code>	50 requests/s	50	60 s	Large in-memory response intended to expose response-path costs and tail instability.

The selected operating points are intentionally fixed rather than derived from a full saturation sweep. This choice reflects the research question: the aim is to compare tail behaviour under controlled offered load, not to rank maximum throughput. The chosen rates were selected to keep the stronger baselines near stable rate attainment while still making queueing effects visible under open-loop load. Lower rates would risk producing artificially flat latency distributions with little comparative value, while substantially higher rates would collapse weaker baselines into overload and reduce interpretability. The operating points therefore represent a compromise between methodological control and discriminatory power rather than an attempt to identify universal saturation thresholds.

6.3 Open-loop load generation

The harness uses `wrk2` rather than closed-loop tools because it can drive a fixed offered rate [7, 13]. Under open-loop load, the client continues issuing requests at the configured rate even if the server slows down. Measured latency therefore includes both service time and any queueing induced by the chosen operating point. That is precisely the behaviour of interest in this dissertation.

The benchmark scripts record the raw `wrk2` output, extract p50, p95, p99, p99.9, request count, and error fields through a Lua metrics script, and append the result to a run-level CSV file together with the benchmark name, URL, target rate, duration, thread count, connection count, and output directory.

6.4 Repetition, order rotation, and aggregation

Each configuration is repeated ten times. The benchmark order is rotated across repetitions rather than fixed, so that a single implementation does not always benefit from warm caches or suffer from late-session thermal or system-noise effects.

The summary tables report a trimmed mean with the two highest and two lowest values removed for each metric. That procedure is only defensible because the raw-run data are retained and examined alongside it. If trimming were presented on its own, it would risk smoothing away meaningful instability. The 50 KiB results for the custom server show exactly why both views are needed: the trimmed summary is competitive, but the raw-run distribution reveals a pronounced outlier.

A 20% trimmed mean is used as a pragmatic summary statistic rather than as a claim that outliers are unimportant. With ten repetitions per configuration, removing the two highest and two lowest runs suppresses isolated disturbances such as transient OS scheduling noise, cache warm-state effects, or unrelated background activity without discarding most of the sample. Median-only reporting

would hide the magnitude of severe disturbed runs, while an untrimmed mean would allow a single pathological repetition to dominate the apparent typical case. For that reason, substantive claims in this report rest on both views simultaneously: trimmed summaries describe typical operating behaviour, while raw distributions and maxima are retained explicitly to qualify stability and expose worst-case behaviour.

No confidence intervals are reported because the study uses only ten repetitions per configuration, which is too small a sample for stable interval estimates once the observed distributions include rare but severe disturbed runs. Instead, the report presents trimmed summaries together with the full raw tables and distribution plots. This is a weaker statistical treatment than a larger repeated study with bootstrap intervals, but it avoids giving a false impression of precision.

6.5 Reproducibility measures

The harness records more than latency numbers. For each run it also stores timestamps, the Git commit if available, `uname` output, file descriptor limits, Stack and GHC versions, the `wrk2` version, and the checksums of the benchmark corpus used on both the main project and baseline side. The curated submission artefacts additionally record toolchain versions such as nginx 1.24.0, Go 1.22.2, Stack 3.7.1, Python 3.12.3, and the Flask/Gunicorn package set used in the final session.

These measures still do not constitute a complete reproducibility package in the strongest systems sense. In particular, the storage medium type and CPU power-governor configuration were not separately recorded, and the same-host topology introduces loopback contention that would not arise in a split client/server deployment. Logging policy was also not fully uniform across baselines, so that detail is reported explicitly in Tables 4 and 5 rather than being smoothed over. Even so, the study is substantially more reproducible than an ad hoc benchmark captured only as a screenshot or a single terminal transcript: the benchmark corpus is checksum-controlled, the launch commands are scripted, the toolchain is recorded, and the raw per-run results are retained in Appendix A.

6.6 Targeted diagnostic pass

The main comparative session answers RQ2, but a later diagnostic pass was added specifically to strengthen interpretation of the 1 MiB weakness. Two benchmark endpoints were used: `/json` as a small-response reference and `/file1m` as the problematic large-response case. The diagnostic harness collected one eventlog-enabled rerun and one profiled rerun for each endpoint, archiving raw `wrk2` output together with RTS event logs, cost-centre reports, and heap profiles.

The diagnostic runs deliberately reused the same offered loads as the main study so that the evidence remained directly comparable in regime even though the binaries differed. The eventlog pass kept the original 60-second duration; the profiled pass used 30 seconds to keep profiling overhead manageable. The eventlog reruns used the normal executable with RTS event logging enabled using `+RTS -N -l-au -RTS`. The profiled reruns used a cost-centre-profiled executable with heap profiling enabled using `+RTS -N -p -hc -i0.01 -RTS`. The diagnostic pass therefore changed the observability configuration of the server, but kept the same benchmark endpoints and offered-load regimes as the main study.

The purpose of this pass is attribution rather than re-ranking implementations. The diagnostic artefacts are therefore used to test the plausibility of the response-path explanation under the same operating regime, not to provide a second fully repeated performance study. For that reason, the profiled binaries are not compared directly against the main-session headline tables, and the diagnostic pass is interpreted as supporting evidence rather than as a replacement result set.

6.7 Threats to validity

The evaluation is necessarily bounded. It is a local, controlled study rather than an internet-scale deployment, so the conclusions concern comparative behaviour on the tested machine and configuration. The reported offered rates are specific operating points rather than a full load sweep, meaning the results say more about behaviour in these regimes than about global capacity limits.

A further limitation is topology. The `wrk2` client and the server under test ran on the same host over loopback, so the measurements include shared-host contention and exclude the effects of a real network path. That choice improves control and repeatability, but it also narrows the external validity of the latency numbers.

Baseline fairness is another bounded aspect of the study. The harness serves byte-identical payloads across all implementations and records the launch configuration used in the final session, but the baselines still differ in abstraction level, tuning surface, and logging policy. `nginx` is best read as a static-serving reference, `Warp` as the closest like-for-like peer, and `Go` and `Flask+Gunicorn` as informative external comparison points rather than identical application-server matches.

Finally, the later diagnostic pass reduces uncertainty without removing every interpretive limit. It is intentionally narrow, it was not repeated ten times, and the present report does not include a separate ThreadScope-based scheduler analysis of the captured event logs. The causal discussion therefore rests most strongly on the cost-centre and heap-profile evidence, with the eventlog artefacts retained as supporting material rather than treated as a complete scheduler-level explanation.

7 Results

Figure 1 summarises trimmed mean p99 latency across workloads and implementations, while Figure 2 shows how closely each system sustained the configured offered rate. Figure 3 then complements the trimmed summaries with raw-run distributions.

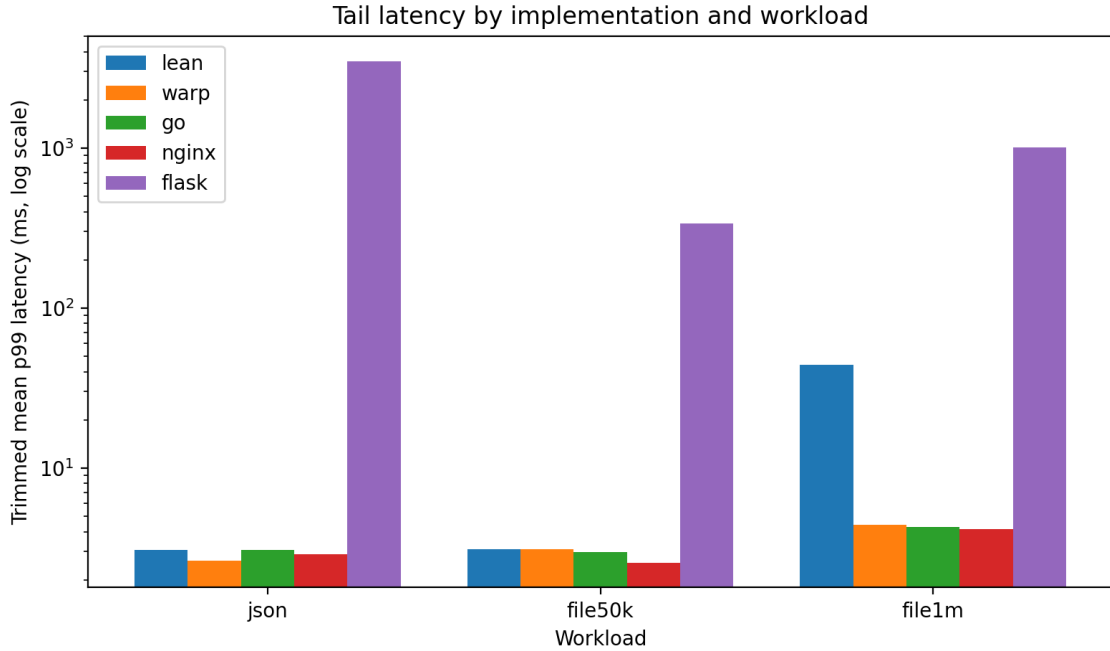


Figure 1: Trimmed mean p99 latency across workloads. The custom server remains in the same general regime as Warp, Go, and nginx on the JSON and 50 KiB workloads, but diverges sharply on the 1 MiB case. A log scale is used because the Flask baseline is orders of magnitude slower than the other implementations.

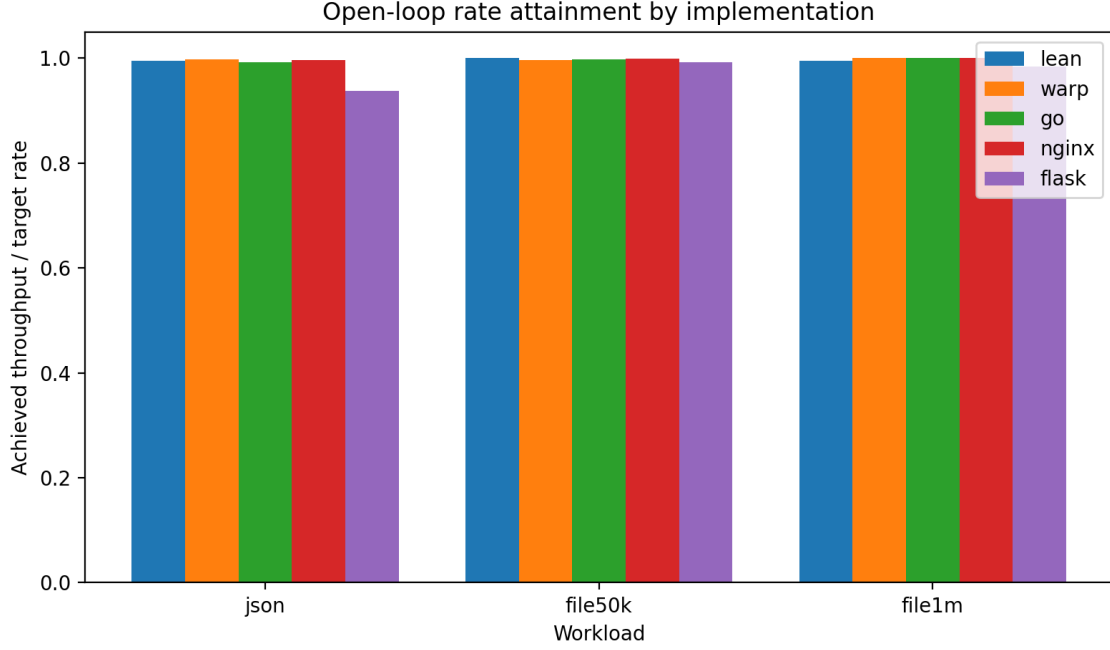


Figure 2: Achieved throughput normalised by target offered rate. Values near 1.0 indicate that the implementation sustained the intended open-loop load. The key point is that the custom server’s 1 MiB weakness is a latency problem rather than a failure to meet the configured demand.



Figure 3: Raw-run p99 distributions by implementation and workload. This makes the stability story harder to overstate: typical behaviour is visible from the boxes, while the long upper tails and isolated outliers remain explicit. A log scale is used to keep the slow Flask regime visible without flattening the faster implementations.

Tables 7–9 report the trimmed aggregates used in the main discussion. In these tables, bold indicates the strongest value in a column and underlining marks the weakest. For Achieved RPS, “strongest” means closest to the target offered rate rather than numerically highest throughput.

Table 7: Trimmed benchmark summary for the 1 KiB JSON workload at a target offered load of 2000 requests/s. The immediate takeaway is that the custom server remains in the same low-single-digit p99 regime as Warp, Go, and nginx.

Implementation	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	1989.14	1.361	2.448	3.083	3.858
warp	1994.15	1.181	2.254	2.632	2.950
go	1983.97	1.325	2.559	3.078	4.106
nginx	1991.55	1.239	2.372	2.896	3.349
flask	<u>1874.02</u>	<u>1943.039</u>	<u>3326.292</u>	<u>3493.887</u>	<u>3536.554</u>

Table 8: Trimmed benchmark summary for the 50 KiB binary workload at a target offered load of 300 requests/s. The custom server is competitive on typical p99 latency, but this table should be read alongside the raw-run distribution because trimming suppresses a substantial outlier.

Implementation	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	300.02	1.521	<u>2.655</u>	3.120	3.670
warp	299.09	1.467	2.591	3.100	<u>4.683</u>
go	299.19	1.466	2.467	2.995	3.621
nginx	299.60	1.326	2.178	2.566	2.996
flask	<u>297.71</u>	<u>249.407</u>	<u>332.884</u>	<u>338.346</u>	<u>339.924</u>

Table 9: Trimmed benchmark summary for the 1 MiB binary workload at a target offered load of 50 requests/s. The decisive result is that the custom server still sustains the target rate, but with a stable order-of-magnitude p99 gap relative to the strongest baselines.

Implementation	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	49.77	6.859	23.937	44.047	48.900
warp	50.03	2.931	4.003	4.400	4.882
go	50.03	2.909	3.826	4.266	4.848
nginx	50.03	2.721	3.763	4.168	4.544
flask	<u>49.23</u>	<u>992.682</u>	<u>1003.775</u>	<u>1005.652</u>	<u>1006.506</u>

7.1 Overall picture

Two broad facts stand out immediately. First, the custom server sustains the intended offered load on all three workloads almost as well as Warp, Go, and nginx. The achieved-rate ratios in Figure 2 stay close to 1.0. Second, rate attainment and tail quality are not the same thing. The custom server can meet the target rate on the 1 MiB workload while still exhibiting much worse p99 latency than the strongest baselines.

That distinction is one of the useful consequences of open-loop measurement. A purely throughput-centred report could have made the 1 MiB case look much healthier than it is. The most direct comparison throughout remains Warp, since it is the closest peer in language and runtime terms; the Go and nginx results are better read as strong external reference points than as identical application-server matches.

7.2 Interpreting the Flask baseline

Flask+Gunicorn occupies a different role from Warp, Go, and nginx in this comparison. It is the only Python/WSGI stack in the baseline set and the only baseline built around a dynamically interpreted application framework rather than a compiled server or static-serving implementation. Its results are not merely weaker than the other baselines; they are genuinely poor in this experimental regime. On the JSON workload, its trimmed mean p99 latency is 3493.887 ms, compared with low-single-digit millisecond p99 values for the stronger implementations. On the 50 KiB and 1 MiB workloads, it remains hundreds of milliseconds to around one second at the tail. This is large enough that the log-scaled figures are not just a presentational convenience but a requirement if the Flask regime is to remain visible alongside the others.

Those results should still be interpreted carefully. They should not be read as a general indictment of Python web deployment or as proof that all interpreted implementations must behave this way. The configured Gunicorn setup uses a single synchronous worker and retains access logging, so it is closer to a deliberately simple illustrative baseline than to a tuned production Flask deployment. The result is nevertheless useful: it shows that, under the same byte-identical corpus and offered-load harness, a simple Python/WSGI application stack can fall into a radically different tail-latency regime from Warp, Go, nginx, and the custom Haskell server. Flask therefore acts as a deliberately weak but informative contrast rather than as a like-for-like competitor.

7.3 JSON workload

For the 1 KiB JSON workload, the custom server records a trimmed mean p99 latency of 3.083 ms while sustaining 1989.14 requests/s against a 2000 requests/s target. This places it essentially level with Go (3.078 ms), around 6% above nginx (2.896 ms), and around 17% above Warp (2.632 ms). The practical conclusion is modest but positive: for a small in-memory response, the direct Haskell implementation remains in the same performance regime as the strongest baselines.

The JSON raw runs are not perfectly flat, but they are not alarming either. The custom server's p99 values span roughly 2.58–3.98 ms across the ten repetitions. That is real variation, but it is a normal kind of variation rather than evidence of collapse or intermittent failure.

7.4 50 KiB workload

For the 50 KiB workload, the custom server achieves 300.02 requests/s and a trimmed mean p99 of 3.120 ms. This is effectively identical to Warp (3.100 ms), about 4% above Go (2.995 ms), and around 22% above nginx (2.566 ms). On the trimmed aggregates alone, the custom server therefore appears competitive for medium-sized payloads.

That conclusion needs qualification. The raw p99 runs for the custom server on this workload are not merely noisy; one repetition reaches 413.183 ms, while several others stay close to 2.5–2.6 ms. Because the summary trims the two highest and two lowest runs, the final mean softens this instability substantially. The correct interpretation is therefore not that the 50 KiB case is solved, but that the server's typical behaviour is competitive while its worst observed behaviour is not.

Figures 4 and 5 are included for exactly this reason. Together they prevent the trimmed summary from carrying more interpretive weight than it deserves. Other implementations also exhibited occasional disturbed runs, but the custom server's 50 KiB outlier is materially larger, while its 1 MiB weakness differs in being stable across runs rather than isolated.

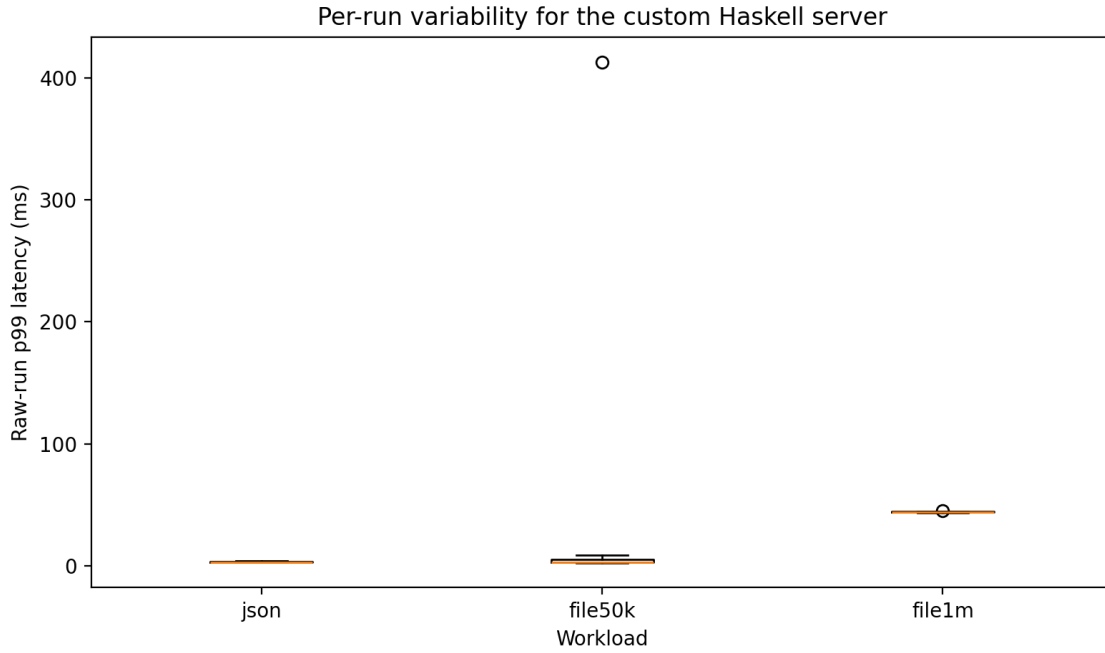


Figure 4: Raw-run p99 variability for the custom Haskell server. The 50 KiB workload shows a pronounced outlier, which is why the trimmed summary must not be read in isolation.

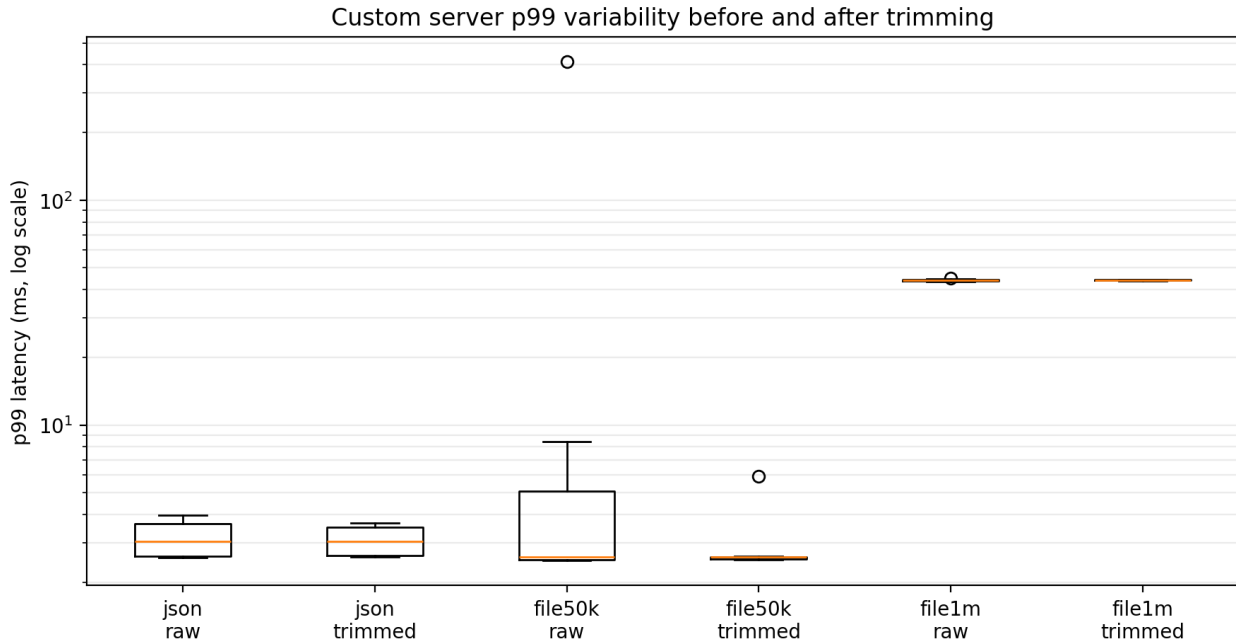


Figure 5: Custom-server p99 variability before and after trimming. The comparison makes visible what the summary statistic is doing: the 50 KiB workload appears much cleaner after trimming, while the 1 MiB weakness remains even when extremes are removed.

7.5 1 MiB workload

The 1 MiB workload is the clearest weakness in the implementation. The custom server still attains the configured rate reasonably closely (49.77 requests/s against a target of 50), but its trimmed mean p99 latency rises to 44.047ms. Warp, Go, and nginx remain near 4–4.4ms. The custom implementation is therefore roughly an order of magnitude worse at p99 on this workload even

though it continues to sustain the offered rate.

Unlike the 50 KiB case, this is not a story of a single bad outlier distorting an otherwise healthy result. The raw p99 values for the custom server on the 1 MiB workload sit tightly in the 43.5–44.9 ms range. In other words, the poor tail behaviour is stable. That makes the result stronger, not weaker: the large gap is a consistent property of this implementation path, not a one-off disturbance.

This is the report’s most important negative finding. It means the dissertation cannot honestly claim broad competitiveness across response sizes. The evidence supports a narrower and more defensible conclusion: the design works well for small and medium in-memory responses, but large-response handling remains the point at which the current architecture stops being competitive.

7.6 Targeted diagnostic pass

The main benchmark tables establish the comparative picture, but the later diagnostic pass is what turns the 1 MiB explanation into something more than a plausible story. Appendix B reports the targeted reruns in full. In brief, the eventlog-enabled reruns preserved the same broad ordering as the main session: `/file1m` again remained in the mid-40 ms p99 regime while `/json` stayed near the low-single-digit millisecond range. The profiled reruns were slower, as expected, with `/file1m` still in the same broad regime and `/json` inflated much more sharply. That asymmetry is itself informative: profiling overhead is relatively modest once the response body is large enough to dominate the request path, but it becomes a much larger fraction of total work when the payload is tiny.

Figure 6 shows the strongest direct attribution evidence from the profiled `/file1m` run. The cost-centre report recorded 215,219,168 bytes of total allocation and assigns 65.6% of time to `sendBuf`, with the response-emission path also prominent in allocation terms: `sendBuilder` accounts for 26.3%, `httpDate` for 16.0%, and `sendLazyChunks` for 12.1% of allocation. `mkResponse` remains present in the hot path, but the dominant picture is not expensive request parsing or repeated disk reads; it is response emission and the allocation structure around it.

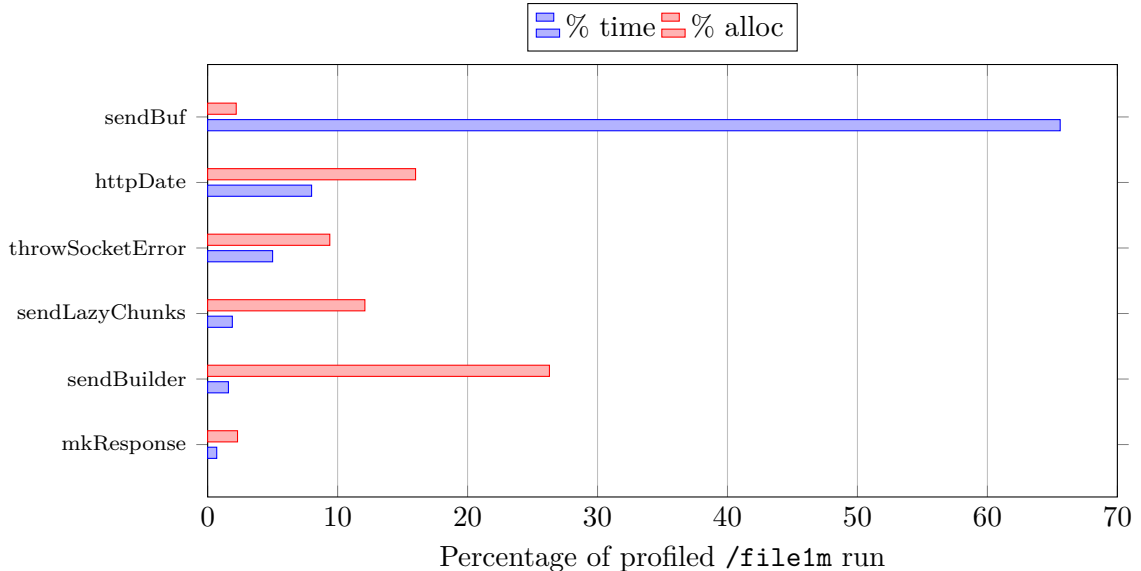


Figure 6: Leading cost centres in the profiled `/file1m` diagnostic run. `sendBuf` dominates execution time, while `sendBuilder`, `httpDate`, and `sendLazyChunks` dominate allocation within the response path. The numerical values are retained in Appendix B.

The heap profile from the same run, shown in Figure 7, reinforces that interpretation. It shows substantial live allocation during the large-response diagnostic run rather than only at startup, which

is consistent with the cost-centre profile’s emphasis on emission and builder-related work. Eventlog artefacts were also captured successfully for both `/json` and `/file1m`; because the present report does not include a separate ThreadScope analysis, they are treated here as supporting artefacts rather than as the primary basis for the causal claim.

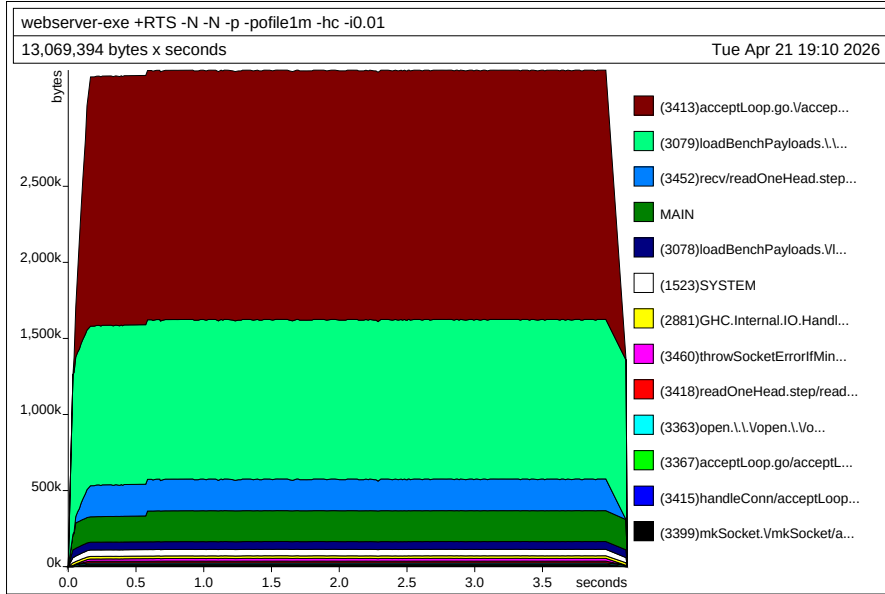


Figure 7: Heap profile from the profiled `/file1m` diagnostic run. The plot corroborates that the large-response path is allocation-active during the period in which the server remains in its high-latency regime.

The purpose of the diagnostic pass is hypothesis testing rather than performance re-ranking. The specific hypothesis under examination is that the 1 MiB weakness is driven primarily by response emission and allocation behaviour rather than repeated disk I/O or unusually expensive request parsing. Because the benchmark payloads are preloaded before the first request is served, repeated disk access is already a weaker explanation; the diagnostic artefacts are used to test whether runtime evidence strengthens or weakens that interpretation. Cost-centre profiling provides the strongest direct attribution, heap profiles indicate whether allocation remains active during the high-latency regime itself, and event-log capture is retained as supporting evidence for later scheduler-level inspection rather than treated as primary proof.

8 Discussion and Critical Reflection

8.1 What the successful cases really show

The JSON and 50 KiB results are encouraging, but they should not be exaggerated. The custom server does not dominate Warp or nginx, and in the JSON case Warp remains clearly better at p99. What the results do show is that explicit request parsing, connection handling, and response construction in Haskell are not so expensive that a lean implementation is pushed out of the

competitive range by default. That is a useful technical result in its own right.

It is also important that these figures are attached to a server with genuine protocol behaviour. The project supports framing, host validation, conditional requests, range handling, and persistent connections, and those features are exercised by the test suite. This does not make the system feature-complete, but it does mean the performance belongs to something more substantial than a benchmark stub.

8.2 Interpreting the 1 MiB weakness from the implementation

The large-response result is where the code becomes analytically useful. The benchmark endpoints are preloaded at startup by `Workloads.loadBenchPayloads`. The 1 MiB gap therefore cannot be dismissed as repeated disk I/O during the benchmark itself. The data are already in memory before the first request arrives. That narrows the plausible causes immediately.

The later diagnostic pass strengthens the explanation considerably. The eventlog-enabled rerun keeps `/file1m` at 44.959 ms p99, which is extremely close to the main-session picture and therefore shows that the weakness survives a separate diagnostic session. The profiled rerun remains in the same broad regime at 46.687 ms p99 despite the expected instrumentation overhead. More importantly, the cost-centre report and heap profile turn the response-path story into something evidenced. In the profiled `/file1m` run, `sendBuf` accounts for 65.6% of time, while `sendBuilder`, `httpDate`, and `sendLazyChunks` account for 26.3%, 16.0%, and 12.1% of allocation respectively. `mkResponse` is still present in the hot path, but the dominant picture is response emission and the allocation structure around it.

That does not prove that any single function is “the bottleneck” in an overly literal sense. The profiled binary perturbs performance, and this report does not include a separate scheduler-level ThreadScope analysis of the captured event logs. Even so, the explanation has moved from plausible implementation narrative to supported interpretation. Because the large payload is preloaded in memory and the strongest direct evidence concentrates in the send and response-construction path, the most defensible interpretation in the present evidence base is that the 1 MiB tail gap is driven primarily by user-space response emission and its interaction with allocation/runtime behaviour rather than by repeated filesystem reads.

8.3 Why the 50 KiB outlier matters methodologically

The 50 KiB case provides a second lesson. If the report had stopped at the trimmed tables, it would have implied a cleaner story than the raw data support. The raw-run outlier above 400 ms is too large to dismiss casually. It may be a transient runtime event, an OS-level disturbance, or something more specific in the send path. At present, the data do not let that be resolved. The cautious interpretation is not to speculate too confidently, but to show the outlier and narrow the claim.

This is where the benchmarking literature is directly relevant. Methodological discipline is not only about choosing open-loop load. It is also about resisting summaries that flatten inconvenient structure in the data [4, 7]. The trimmed mean remains a useful summary, but only because the report also makes visible what trimming hides. The diagnostic pass is intentionally narrow: it is designed to test the plausibility of the response-path explanation under the same operating regime, not to provide a second fully repeated performance study.

8.4 Suitability of Haskell for this task

Haskell was well suited to the correctness-oriented parts of the project. Request parsing, target validation, body-framing decisions, precondition evaluation, and range handling benefited from being

written as small functions over explicit data types. Referential transparency made many of these paths easier to reason about because parsing and decision logic could be separated from socket effects. This was particularly useful in keeping protocol policy visible: malformed framing, invalid targets, and conditional request outcomes could be represented as ordinary values before being turned into wire responses.

The same strengths were less decisive in the performance-critical response path. Once the bottleneck moved from protocol decision-making to large-payload emission, the useful questions became less about equational reasoning and more about allocation, strictness, buffer boundaries, syscall behaviour, and RTS scheduling. In that part of the project, idiomatic clarity was not enough. Achieving better tail latency would likely require more specialised `ByteString` handling, reduced builder allocation, date-header caching, streaming or lower-copy response emission, and systematic RTS tuning.

Debugging followed the same split, and this is one of the more important practical qualifications to the suitability claim. At the semantic level, functional programming helped: pure parsing and decision functions were often easier to reason about than equivalent code interleaved with mutation and socket effects. At the operational level, however, debugging was not always easier. Haskell is a less common paradigm for low-level web-server implementation than imperative systems programming, so there is a smaller pool of familiar debugging patterns, examples, and performance folklore to draw on. Laziness and allocation behaviour can also make the route from source code to runtime cost less direct than it first appears.

Functional correctness bugs were generally local and inspectable, especially in parsing and filesystem semantics. Performance debugging was harder: the 1 MiB weakness was not obvious from the source alone and only became convincing after cost-centre profiling, heap profiling, and event-log capture. This suggests that Haskell is a credible language for this kind of server, but that extracting competitive tail latency from large response paths requires systems-level tuning that partly cuts against the simplicity that made the initial design attractive. In short, Haskell made some correctness arguments cleaner, but it did not make the whole engineering process simpler.

8.5 Limitations of the present design

Several design limitations are worth stating plainly.

First, response construction is simple rather than aggressively optimised. That makes the implementation easy to inspect, but it also means that large-payload efficiency is unlikely to match a mature server whose response path has seen years of tuning.

Second, the benchmark study uses fixed operating points rather than a full load sweep. This makes the final report readable and the benchmark session manageable, but it also means the dissertation cannot say much about collapse thresholds, saturation curves, or the precise relationship between offered load and instability.

Third, the server is standards-aware only for a scoped subset. It is not intended as a production-ready replacement for Warp.

8.6 Future work

The strongest candidate is a more streaming-oriented or lower-copy large-response path that reduces work in `sendBuf`, `sendBuilder`, and `sendLazyChunks`. A second low-cost improvement would be to cache or amortise `httpDate` generation rather than rebuilding the date header for every response, since it appears prominently in both diagnostic profiles.

A further extension would be a fuller load sweep. A broader offered-load curve would make it possible to separate stable-load behaviour from overload more explicitly and to show where instability begins

rather than only where it is already visible.

Finally, the captured eventlog artefacts should be analysed directly in ThreadScope, and the same implementation should be compared under varied RTS settings. The executables are already built with `-threaded` and `-rtsopts`; exploiting that more systematically would connect the empirical story more directly to the GHC runtime literature and help distinguish response-path cost from scheduler behaviour.

Table 10: Closure against the research questions and success criteria.

RQ	Success criterion	Evidence	Outcome
RQ1	Correct behaviour for the scoped HTTP/1.1 subset.	53 socket-level end-to-end tests covering request parsing, framing, host validation, filesystem operations, conditional requests, range handling, and persistent connections.	Met within the stated scope.
RQ2	Competitive tail latency at fixed offered load, especially on smaller responses.	Open-loop <code>wrk2</code> results against Warp, Go <code>net/http</code> , nginx, and Flask+Gunicorn using shared payload bytes, ten repetitions, and retained raw runs.	Met for 1 KiB and broadly met on typical 50 KiB behaviour; not met for 1 MiB.
RQ3	A practical and honest account of where the design works and where it does not.	Comparative results, raw-run distributions, implementation analysis, Haskell-specific suitability reflection, and explicit discussion of methodological and architectural limits.	Met as a bounded systems study rather than as a production-readiness claim.

9 Conclusion

This dissertation set out to determine whether a lean Haskell HTTP/1.1 server could be made sufficiently correct and sufficiently performant to support a meaningful tail-latency study. Within its stated scope, the answer is yes. The server implements a non-trivial subset of HTTP/1.1 semantics and supports that claim with substantial socket-level testing. Under fixed-rate open-loop load, it remains close to Warp, Go, and nginx on the 1 KiB JSON and 50 KiB workloads, which shows that the overall implementation strategy is technically credible.

At the same time, the 1 MiB results establish a clear limit. The present design does not yet offer strong large-payload tail behaviour, and the gap is large enough that it cannot be written off as measurement noise. The later diagnostic pass substantially narrows that explanation: the weakness remains stable under rerun, and the strongest direct evidence concentrates in the response-emission path rather than in repeated disk reads. That limitation gives the dissertation a concrete technical result: it identifies the boundary at which a simple, direct Haskell implementation ceases to be competitive and therefore the point at which future optimisation effort should be concentrated. The comparative results establish the bounded competitiveness claim directly; the targeted diagnostic pass strengthens, but does not fully close, the causal explanation of the large-payload weakness.

The main boundary result is therefore that the simple design holds up for smaller responses, but not for the large-response path. The project shows that direct systems implementation in Haskell can remain both standards-aware and performance-credible without requiring framework-scale abstraction, but it also identifies precisely where that approach becomes difficult to defend: large-payload response paths where allocation structure and emission cost dominate tail behaviour. That boundary is the main technical contribution of the dissertation. Rather than claiming universal competitiveness, it establishes where a lean implementation remains viable, where it does not, and where future optimisation effort must be concentrated.

References

- [1] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.
- [2] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” Internet Engineering Task Force, RFC 9110, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>. Accessed: Apr. 28, 2026.
- [3] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP/1.1,” Internet Engineering Task Force, RFC 9112, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9112>. Accessed: Apr. 28, 2026.
- [4] G. Heiser, “Systems Benchmarking Crimes,” 2010. [Online]. Available: <https://gernot-heiser.org/benchmarking-crimes.html>. Accessed: Apr. 28, 2026.
- [5] S. Marlow, “Developing a high-performance web server in Concurrent Haskell,” *Journal of Functional Programming*, vol. 12, no. 4–5, pp. 359–374, Jul. 2002.
- [6] B. O’Sullivan and J. Tibell, “Scalable I/O event handling for GHC,” in *Proc. ACM SIGPLAN Haskell Symp.*, Baltimore, MD, USA, 2010, pp. 103–108.
- [7] G. Tene, “How NOT to Measure Latency,” presented at QCon London, London, U.K., 2013. [Online]. Available: <https://www.infoq.com/presentations/latency-pitfalls/>. Accessed: Apr. 28, 2026.
- [8] A. R. Voellmy, J. Wang, P. Hudak, and K. Yamamoto, “Mio: A high-performance multicore IO manager for GHC,” in *Proc. ACM SIGPLAN Haskell Symp.*, Boston, MA, USA, 2013, pp. 129–140.
- [9] K. Yamamoto, M. Snoyman, and A. Voellmy, “Warp: A Haskell web server,” in *The Performance of Open Source Applications*, A. Brown and G. Wilson, Eds. AOSA Project, 2013.
- [10] Hackage, “attoparsec: Fast combinator parsing for bytestrings and text,” package documentation. [Online]. Available: <https://hackage.haskell.org/package/attoparsec>. Accessed: Apr. 28, 2026.
- [11] Hackage, “wai: Web Application Interface,” package documentation. [Online]. Available: <https://hackage.haskell.org/package/wai>. Accessed: Apr. 28, 2026.
- [12] Hackage, “warp: A fast, light-weight web server for WAI applications,” package documentation. [Online]. Available: <https://hackage.haskell.org/package/warp>. Accessed: Apr. 28, 2026.
- [13] G. Tene *et al.*, “wrk2: A constant throughput, correct latency recording variant of wrk,” GitHub repository. [Online]. Available: <https://github.com/giltene/wrk2>. Accessed: Apr. 28, 2026.

A Raw benchmark tables

These appendix tables provide the full per-run metrics underlying the trimmed summaries reported in the body. They are included for auditability and completeness; the main argumentative claims of the dissertation continue to rest on the self-contained evidence presented in the report body.

Table 11: Raw per-run benchmark results for the 1 KiB JSON workload.

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	1	1981.64	1.444	2.923	3.977	5.375
lean	2	1996.65	1.259	2.217	2.581	2.929
lean	3	1981.64	1.427	2.705	3.671	4.995
lean	4	1981.62	1.450	2.705	3.519	4.567
lean	5	1996.65	1.237	2.257	2.633	3.053
lean	6	1996.62	1.312	2.191	2.595	2.907
lean	7	1996.66	1.328	2.185	2.579	2.923
lean	8	1981.63	1.572	2.889	3.763	5.247
lean	9	1981.65	1.393	2.609	3.443	4.663
lean	10	1996.65	1.256	2.197	2.637	2.943
warp	1	1981.65	1.393	2.887	3.655	4.331
warp	2	1981.70	1.506	3.369	4.407	5.239
warp	3	1996.69	1.136	2.081	2.475	2.699
warp	4	1996.62	1.183	2.185	2.461	2.677
warp	5	1996.65	1.150	2.155	2.441	2.697
warp	6	1981.64	1.331	2.727	3.463	4.171
warp	7	1996.69	1.081	2.095	2.383	2.715
warp	8	1996.65	1.136	2.073	2.439	2.723
warp	9	1996.64	1.126	2.221	2.489	2.697
warp	10	1996.64	1.153	2.143	2.465	2.653
go	1	1995.44	1.100	2.021	2.445	4.215
go	2	1981.67	1.368	2.597	3.119	3.967
go	3	1981.63	1.311	2.661	3.249	4.675
go	4	1981.65	1.362	2.681	3.245	3.941
go	5	1996.66	1.165	2.073	2.481	2.735
go	6	1996.65	1.174	2.191	2.521	2.729
go	7	1975.57	1.463	3.005	4.051	6.523
go	8	1981.72	1.362	2.621	3.169	4.387
go	9	1981.70	1.373	2.639	3.183	3.829
go	10	1981.64	1.371	2.647	3.229	4.299
nginx	1	1996.22	1.134	2.149	2.497	2.831
nginx	2	1979.64	1.377	2.763	3.711	4.407
nginx	3	1996.61	1.189	2.157	2.509	2.791
nginx	4	1981.60	1.486	2.885	3.655	4.275
nginx	5	1981.58	1.392	2.791	3.721	4.391
nginx	6	1996.62	1.191	2.121	2.475	2.763
nginx	7	1996.60	1.086	2.065	2.435	2.925
nginx	8	1981.63	1.421	2.937	3.759	4.387
nginx	9	1996.63	1.150	2.181	2.499	2.869
nginx	10	1996.63	1.130	2.191	2.505	2.805
flask	1	1875.16	2238.463	3528.703	3600.383	3639.295
flask	2	1864.82	2115.583	3870.719	3917.823	3958.783
flask	3	1879.43	1801.215	3311.615	3467.263	3512.319

Continued on next page

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
flask	4	1868.15	2629.631	3758.079	3837.951	3862.527
flask	5	1887.17	1836.031	2994.175	3244.031	3289.087
flask	6	1880.10	2044.927	3323.903	3485.695	3506.175
flask	7	1876.46	1941.503	3327.999	3553.279	3606.527
flask	8	1809.80	1882.111	3037.183	3225.599	3278.847
flask	9	1911.55	1700.863	2412.543	2529.279	2559.999
flask	10	1845.33	1838.079	3428.351	3612.671	3665.919

Table 12: Raw per-run benchmark results for the 50 KiB binary workload.

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	1	300.02	1.505	2.279	2.591	3.089
lean	2	300.02	1.517	2.203	2.525	3.043
lean	3	300.02	1.521	2.199	2.515	2.881
lean	4	300.02	1.516	2.187	2.501	2.821
lean	5	300.02	1.519	2.207	2.499	2.931
lean	6	300.02	1.504	2.271	2.599	2.869
lean	7	300.02	1.546	99.711	413.183	731.135
lean	8	297.56	1.965	4.691	5.903	7.159
lean	9	300.02	1.487	2.281	2.589	2.915
lean	10	297.55	2.165	6.919	8.439	10.375
warp	1	300.02	1.355	2.045	2.357	2.657
warp	2	299.60	1.354	2.109	2.421	12.007
warp	3	300.02	1.364	2.036	2.375	2.705
warp	4	297.53	1.678	3.663	4.603	5.507
warp	5	297.52	1.704	3.769	4.643	6.375
warp	6	300.02	1.360	2.022	2.343	2.625
warp	7	297.52	1.683	3.579	4.379	5.135
warp	8	299.84	1.363	2.157	2.595	19.871
warp	9	297.52	1.683	3.563	4.415	5.695
warp	10	300.02	1.348	2.095	2.413	2.681
go	1	300.02	1.372	2.155	2.453	2.697
go	2	300.02	1.378	2.047	2.361	2.679
go	3	297.52	1.617	3.225	4.191	5.339
go	4	300.02	1.378	2.051	2.343	2.653
go	5	300.02	1.382	2.049	2.359	2.661
go	6	300.02	1.383	2.040	2.357	2.661
go	7	297.52	1.664	3.317	4.643	7.175
go	8	300.02	1.349	2.015	2.349	2.745
go	9	297.54	1.686	3.663	4.523	26.239
go	10	297.53	1.658	3.273	4.251	5.603
nginx	1	300.02	1.285	2.023	2.325	2.619
nginx	2	300.02	1.279	1.975	2.311	2.607
nginx	3	300.02	1.269	1.966	2.275	2.543
nginx	4	297.52	1.550	3.051	3.823	5.007
nginx	5	300.02	1.281	1.985	2.303	2.545
nginx	6	297.52	1.606	3.283	4.037	5.199
nginx	7	300.02	1.279	1.982	2.297	2.555
nginx	8	300.02	1.242	2.005	2.315	2.613
nginx	9	297.52	1.638	3.473	4.271	5.355
nginx	10	300.02	1.280	2.023	2.319	2.577
flask	1	298.38	248.959	325.631	332.031	332.799
flask	2	296.28	248.319	349.695	353.535	357.887
flask	3	298.38	251.135	326.911	333.311	334.079
flask	4	298.38	249.343	326.399	333.055	333.823
flask	5	296.26	253.823	350.463	354.815	357.375
flask	6	296.46	193.663	340.991	343.551	345.599
flask	7	298.38	249.343	326.399	333.567	334.335
flask	8	298.74	2.203	4.759	10.439	143.871
flask	9	298.38	249.343	326.911	333.055	334.335

Continued on next page

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
flask	10	296.18	278.783	354.303	358.143	361.215

Table 13: Raw per-run benchmark results for the 1 MiB binary workload.

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
lean	1	50.03	3.281	5.015	43.903	45.535
lean	2	49.64	8.983	33.727	43.871	51.679
lean	3	50.03	3.303	5.383	44.255	46.143
lean	4	50.03	3.289	5.071	44.223	45.663
lean	5	49.61	8.559	32.895	43.839	57.567
lean	6	49.64	8.351	32.927	43.551	48.607
lean	7	49.62	8.663	33.183	43.455	46.751
lean	8	49.63	10.215	34.431	44.863	67.007
lean	9	49.64	9.071	33.855	44.191	54.367
lean	10	50.03	3.293	5.507	44.511	45.855
warp	1	50.03	2.879	3.891	4.311	5.283
warp	2	50.03	2.977	4.055	4.463	4.803
warp	3	50.03	2.929	3.999	4.351	4.651
warp	4	50.03	2.899	3.745	4.039	4.687
warp	5	50.03	2.915	3.993	4.447	4.867
warp	6	50.03	3.005	4.127	4.443	4.771
warp	7	50.03	2.971	4.111	4.523	4.779
warp	8	50.03	3.043	14.287	46.143	191.615
warp	9	50.03	2.833	3.803	4.163	17.711
warp	10	50.03	2.893	3.971	4.387	4.791
go	1	50.03	2.979	3.875	4.259	4.627
go	2	50.03	2.827	3.751	4.231	4.663
go	3	50.03	2.921	3.775	4.195	4.847
go	4	50.03	2.965	3.913	4.343	4.703
go	5	50.03	2.843	3.845	4.323	5.367
go	6	50.03	2.793	3.851	4.371	4.887
go	7	50.03	2.845	3.793	4.219	4.695
go	8	50.03	2.937	3.767	4.187	4.947
go	9	49.61	6.695	28.991	31.791	40.607
go	10	50.03	2.945	3.815	4.219	5.007
nginx	1	50.03	2.745	3.831	4.255	4.551
nginx	2	50.03	2.795	3.855	4.223	4.635
nginx	3	50.03	2.791	3.845	4.223	4.615
nginx	4	50.03	2.755	3.823	4.183	4.495
nginx	5	50.03	2.583	3.465	3.915	4.463
nginx	6	49.61	5.795	23.535	26.159	27.823
nginx	7	50.03	2.535	3.481	3.875	4.291
nginx	8	50.03	2.689	3.789	4.207	4.555
nginx	9	50.03	2.763	3.807	4.339	4.559
nginx	10	50.03	2.575	3.449	3.901	4.491
flask	1	49.23	993.791	1005.055	1009.151	1025.023
flask	2	49.23	992.767	1004.543	1006.079	1007.103
flask	3	49.23	990.719	1003.519	1005.055	1006.079
flask	4	49.23	993.279	1004.031	1006.079	1007.103
flask	5	49.23	992.255	1004.031	1005.567	1006.079
flask	6	49.23	992.767	1003.519	1005.055	1005.567
flask	7	49.23	994.303	1003.519	1005.567	1006.591
flask	8	49.49	25.391	52.767	59.679	68.991
flask	9	49.23	992.767	1004.031	1005.567	1006.591

Continued on next page

Implementation	Run	Achieved RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)
flask	10	49.22	992.255	1003.519	1006.079	1006.591

B Diagnostic evidence

The tables in this appendix summarise the later diagnostic pass used to strengthen the interpretation of the 1 MiB weakness.

Table 14: Diagnostic reruns captured through the dedicated eventlog and profiling harness.

Run type	Endpoint	Duration (s)	Achieved RPS	p99 (ms)	p99.9 (ms)
eventlog	/file1m	60	49.62	44.959	64.927
eventlog	/json	60	1981.73	3.927	5.211
profile	/file1m	30	50.05	46.687	51.167
profile	/json	30	1933.37	12.431	15.911

Table 15: Leading /file1m cost centres from the profiled diagnostic run.

Cost centre	% time	% alloc
sendBuf	65.6	2.2
httpDate	8.0	16.0
throwSocketError	5.0	9.4
sendLazyChunks	1.9	12.1
sendBuilder	1.6	26.3
mkResponse	0.7	2.3